2.9 Java 8 Lambda and Stream API

***Functional Interface*:**

A *functional interface* is an interface that contains exactly *one abstract method*. Such interfaces may contain **multiple** *default* or *static* methods, but only a **single** *abstract* method defines their functional nature.

Examples from the Java standard library:

- *java.lang.**Runnable*** → void ***run***()
- *java.util.concurrent.**Callable**<V>* → V ***call***()
- *java.util.function.**Predicate**<T>* → *boolean* ***test***(T *t*)
- *java.util.function.**Function**<T, R>* → *R* ***apply***(T *t*)

```
@FunctionalInterface
interface DoubleNumber {
   int work(int x);
}
```

Functional interfaces serve as the *target type* for *lambda expressions* and *method references*. They can be annotated with *@FunctionalInterface* (optional but recommended). This annotation helps the compiler enforce the "**single abstract method**" rule.

***Instance creation of a functional interface*:**

We can create an instance of functional interface in *four ways*,

- *implementing* the interface (*concrete class*)
- using an *Anonymous Inner Class*
- using a *Lambda Expression*
- using *Method Reference*

Concretely class

```
class DoubleValue implements DoubleNumber {
   @Override
   public int work(int x) {
      return x * 2;
   }
} //implement in concrete class and use that obj.
```

Anonymous inner class

```
DoubleNumber w2 = new DoubleNumber() {
   @Override
   public int work(int x) {
      return x * 2;
   }
}; // implement method while object creation.
```

***Lambda Expression*:**

- A *lambda expression* is a concise way to represent an *implementation* of a *functional interface*.
- It provides a **clear** and **readable syntax** for writing instances of functional interfaces, eliminating boilerplate code (such as anonymous classes).

Different variations of lambda expression:

```
DoubleNumber w = (int x) -> {
   return x * 2;
};
```

```
DoubleNumber w = (int x) -> x * 2;
```

```
DoubleNumber w = (x) -> x * 2;
```

```
DoubleNumber w = x -> x * 2;
```

***Method References in Java*:**

A *method reference* in Java is a shorthand notation of a *lambda expression* that calls an *existing method*. It provides a clean and readable way to *refer* to *methods without explicitly invoking them*.

- Lambda Expression**:** (x) -> ClassName.methodName();
- Method Reference **:** ClassName **: :** methodName;

***Purpose*:**

- To reuse *already existing methods* instead of *duplicating logic* inside a *lambda expression*.
- To improve code *readability* and *conciseness*.
- To make *functional programming* in Java (with *Streams*, *Functional Interfaces*) more *expressive*.

***Method Reference Requirement*:**

- A *method reference* can be used **only** with *functional interfaces*.
- That is because method references map to the *single abstract method* (SAM) of the functional interface.

Example:

```
@FunctionalInterface
interface A {
   int sum(int a, int b);
}
```

Lambda-> calling existing Method

```
public static void main(String[] args) {
   A a = (x, y) -> {
      return Integer.sum(x, y);
   };
   System.out.println(a.sum(5, 5));
} // output: 10
```

Method reference

```
public static void main(String[] args) {
   A a = Integer::sum;
   System.out.println(a.sum(5, 5));
} // output: 10
```

here sum(i, j) of A referring to sum(i, j) of Integer class using method reference.

@Sibasundarj8

Types of Method References:

Type	Syntax	Description	Example
Static Method Reference	ClassName::**static**Method	Refers to a ***static method*** of a class	Integer::parseInt
Instance Method Reference of a Particular Object	**object**::**instance**Method	Refers to an ***instance method*** (non-static) of a ***specific object***	myPrinter::print
Instance Method Reference of an Arbitrary Object of a Class	ClassName::**instance**Method	Refers to an ***instance method*** of objects of a ***particular type***	String::toUpperCase
Constructor Reference	ClassName::**new**	Refers to a constructor	ArrayList::new

Example of Constructor reference:

```
class S {
    String name;
    int age;

    S() {} // constructors
    S(String name) {
        this.name = name;
    }
    S(int age) {
        this.age = age;
    }
    S(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @Override
    public String toString() {
        return "Name: " + name + "\n" +
               "Age: " + age + "\n";
    }
}
```

```
/// main Method
public static void main(String[] args) {
    Supplier<S> o1 = S::new;
    System.out.println(o1.get());

    Function<String, S> o2 = S::new;
    System.out.println(o2.apply("Sibasundar Jena"));

    Function<Integer, S> o3 = S::new;
    System.out.println(o3.apply(22));

    BiFunction<String, Integer, S> o4 = S::new;
    System.out.println(o4.apply("Sibasundar Jena", 22));
}
```

Output:

```
Name: null
Age: 0

Name: Sibasundar Jena
Age: 0

Name: null
Age: 22

Name: Sibasundar Jena
Age: 22
```

Stream API:

- The **Stream API** (introduced in ***Java 8***) is a powerful feature that allows developers ***to process collections*** of ***objects*** in a ***functional*** and ***declarative*** style.
- It provides a ***sequence of elements*** supporting ***operations*** that can be ***pipelined*** to produce desired results (such as ***filtering***, ***mapping***, ***reducing***).
- **Key point:** Stream API is not about ***I/O streams*** (like **InputStream/OutputStream**). It is about ***processing data*** in ***collections*** such as ***List***, ***Set***, or ***arrays***.

Characteristics:

- **Functional style** → Focus on ***what to do*** rather than ***how to do***.
- **Pipeline processing** → Operations are ***chained together***.
- **Lazy evaluation** → ***Intermediate*** operations are ***executed only*** when a ***terminal operation*** is ***invoked***.
- **Can be parallelized** → Supports ***parallel streams*** to utilize ***multi-core processors***.

Stream API Operations:

1. ***Intermediate Operations:*** (Return a *new Stream*, *lazy evaluation*, can be *chained together*)
   - map(*Function*) → Transforms each element.
   - mapToInt(*ToIntFunction*) → Maps elements to *IntStream*.
   - mapToLong(*ToLongFunction*) → Maps elements to *LongStream*.
   - mapToDouble(*ToDoubleFunction*) → Maps elements to *DoubleStream*.
   - flatMap(*Function*) → Flattens *nested streams* into a *single stream*.
   - flatMapToInt(*Function*), flatMapToLong(*Function*), flatMapToDouble(*Function*) → Flat mapping for *primitive* streams.
   - filter(*Predicate*) → Filters elements by *condition*.
   - distinct() → Removes *duplicate elements* (based on *equals()*).
   - sorted() → Sorts elements in *natural order*.
   - sorted(*Comparator*) → Sorts elements using *custom comparator*.
   - peek(*Consumer*) → Performs an *action* on each element (for debugging/logging).
   - limit(long *n*) → Truncates stream to given number of elements.
   - skip(long *n*) → Skips *first n* elements.
   - takeWhile(*Predicate*) *(Java 9+)* → Takes elements while *condition holds*.
   - dropWhile(*Predicate*) *(Java 9+)* → Drops elements while *condition holds*, *processes* the *rest*.
   - unordered() → Removes *ordering constraint*.
2. ***Terminal Operations:*** (Produce a *result*, *ends* the *stream pipeline*, *can't* be *reused*)
   - forEach(*Consumer*) → Performs *action* for *each element*.
   - forEachOrdered(*Consumer*) → Like *forEach*, but respects encounter *order*.
   - toArray() → Converts *stream* to *array*.
   - reduce(*BinaryOperator*) → Reduces *elements* into *single result*.
   - reduce(*identity*, *BinaryOperator*) → Reduction with *identity*.
   - collect(*Collector*) → Collects elements into *List*, *Set*, *Map*, etc.
   - min(*Comparator*) → Finds *minimum element*.
   - max(*Comparator*) → Finds *maximum element*.
   - count() → Returns *number of elements*.
   - anyMatch(*Predicate*) → Returns *true* if *any element* matches condition.
   - allMatch(*Predicate*) → Returns *true* if *all elements* match condition.
   - noneMatch(*Predicate*) → Returns *true* if *no element* matches condition.
   - findFirst() → Returns *first element* (wrapped in *Optional*).
   - findAny() → Returns *any element* (useful in *parallel streams*).

Example:

There is a *List* of Strings which contains multiple *names*, we have to *filter* all the names which *starts* with 'S' and *ends* with '*a*' and then return the *max length*.

Without Stream (using loop)

```
public static void main(String[] args) {
    List<String> list = List.of(
            "Siba", "Rahul", "Tushar",
            "Upendra", "Jaga", "Subrat",
            "Kaylash", "Bikram", "Satya",
            "Sibaji", "Sanjay"
    );
    int ans = 0;
    for (String s : list) {
        if (s.startsWith("S") && s.endsWith("a")) {
            ans = Math.max(ans, s.length());
        }
    }

    System.out.println(ans);
}
```

Using Stream

```
public static void main(String[] args) {
    List<String> list = List.of(
            "Siba", "Rahul", "Tushar",
            "Upendra", "Jaga", "Subrat",
            "Kaylash", "Bikram", "Satya",
            "Sibaji", "Sanjay"
    );

    int ans = list.stream()
            .filter(s -> s.startsWith("S") && s.endsWith("a"))
            .map(String::length)
            .max(Integer::compareTo)          // code is more cleaner
            .orElse(0);                       // and readable.

    System.out.println(ans);
}
```